

INFRAESTRUCTURA Y OPERACIONES

Roles, tecnologías, herramientas y ruta de aprendizaje en DevOps

Objetivo de la guía

Explicar qué hace cada perfil de Infraestructura y Operaciones, qué tecnologías utiliza, cómo se relacionan los roles y en qué orden conviene aprender las herramientas.

**DevOps Engineer • Cloud Engineer • Platform Engineer
Site Reliability Engineer • System Administrator • DBA**

Guía educativa - 2026

Mapa visual de Infraestructura y Operaciones

Tecnologías y herramientas principales por rol



La infografía resume la base común y las especializaciones desarrolladas en las páginas siguientes.

Contenido de la guía

1	1. Panorama general Qué comprende Infraestructura y Operaciones.
2	2. Base común Linux, redes, Git, automatización, formatos, seguridad y troubleshooting.
3	3. Roles profesionales DevOps, Cloud, Platform, SRE, System Administrator y DBA.
4	4. Comparación entre roles Objetivos, responsabilidades, herramientas y puntos de contacto.
5	5. Terraform en cada perfil Dónde aporta más valor y cómo se integra con otras herramientas.
6	6. Ruta de aprendizaje Secuencia desde fundamentos hasta especialización.
7	7. Proyectos prácticos Ejercicios de portafolio para demostrar competencias.

Idea central

Estos roles no trabajan aislados. Comparten fundamentos y colaboran sobre la misma plataforma tecnológica, pero cada uno prioriza una pregunta diferente: entregar software, construir cloud, facilitar a desarrolladores, asegurar confiabilidad, administrar sistemas o proteger datos.

¿Qué abarca Infraestructura y Operaciones?

Es el conjunto de prácticas, personas y tecnologías que mantienen disponibles los entornos donde se ejecuta el software. Incluye servidores, redes, nube, contenedores, sistemas operativos, almacenamiento, bases de datos, observabilidad, seguridad y automatización.

Área	Tecnologías y herramientas
Infraestructura	Servidores físicos y virtuales, redes, almacenamiento, balanceadores y servicios cloud.
Operaciones	Monitoreo, administración, mantenimiento, respuesta a incidentes, backups y recuperación.

Área	Tecnologías y herramientas
Automatización	Scripts, CI/CD, infraestructura como código y gestión de configuración.
Confiabilidad	Disponibilidad, rendimiento, SLO, alertas, runbooks y postmortems.
Seguridad	Identidades, permisos, secretos, cifrado, vulnerabilidades y auditoría.

Base común para todos los perfiles

Antes de elegir una especialización, conviene dominar un conjunto compartido de conocimientos. Esta base permite entender el sistema completo y diagnosticar fallas sin depender exclusivamente de herramientas gráficas.

Área	Tecnologías y herramientas
Linux y sistemas operativos	Ubuntu, Debian, RHEL/Rocky/AlmaLinux, Windows Server; procesos, servicios, usuarios, permisos, paquetes y logs.
Redes	TCP/IP, DNS, DHCP, HTTP/HTTPS, puertos, routing, NAT, firewalls, VPN, balanceadores y troubleshooting.
Terminal y scripting	Bash, PowerShell y Python para automatizar tareas repetitivas.
Control de versiones	Git, GitHub, GitLab o Bitbucket; ramas, commits, pull requests, tags y releases.
Configuración	YAML, JSON, HCL, archivos .env y plantillas.
Seguridad	SSH, TLS, certificados, secretos, mínimo privilegio, IAM y gestión de vulnerabilidades.
Observabilidad	Logs, métricas y trazas; dashboards, alertas y análisis de incidentes.
Troubleshooting	Verificar hipótesis con datos, aislar causas, revisar cambios recientes y documentar soluciones.

Comandos que conviene conocer

Área	Tecnologías y herramientas
Procesos y servicios	ps, top, htop, systemctl, journalctl, tasklist, Get-Service.
Red	ping, traceroute, dig, nslookup, curl, ss, netstat, tcpdump y Wireshark.
Archivos y permisos	ls, find, grep, chmod, chown, tar, rsync y PowerShell.
Contenedores	docker ps, docker logs, docker inspect, kubectl get, kubectl describe y kubectl logs.

Área	Tecnologías y herramientas
Git	clone, status, add, commit, branch, merge, pull, push, tag y revert.

Regla práctica

Aprender herramientas sin comprender Linux, redes y procesos produce conocimientos frágiles. La base técnica permite transferir lo aprendido entre AWS, Azure, Google Cloud, Kubernetes y diferentes plataformas.

1. DevOps Engineer

Conecta desarrollo y operaciones. Automatiza el recorrido del código desde el repositorio hasta los entornos de prueba y producción.

Responsabilidades principales

- **Automatización:** reducir tareas manuales y repetitivas mediante pipelines, scripts y plantillas.
- **CI/CD:** compilar, probar, analizar, empaquetar y desplegar aplicaciones de manera controlada.
- **Infraestructura como código:** crear y modificar recursos con Terraform, OpenTofu, CloudFormation, Bicep o Pulumi.
- **Contenedores:** construir imágenes, gestionar registros y desplegar aplicaciones en Kubernetes.
- **Observabilidad y seguridad:** incorporar métricas, logs, escaneo de vulnerabilidades y secretos al ciclo de entrega.

Área	Tecnologías y herramientas
Versionado	Git, GitHub, GitLab, Bitbucket.
CI/CD	GitHub Actions, GitLab CI/CD, Jenkins, Azure DevOps Pipelines, CircleCI.
Contenedores	Docker, Podman, Docker Compose, Kubernetes, Helm, Kustomize.
IaC	Terraform, OpenTofu, Pulumi, CloudFormation, Azure Bicep.
Configuración	Ansible, Puppet, Chef, Salt.
Observabilidad	Prometheus, Grafana, Loki, ELK/OpenSearch, OpenTelemetry, Jaeger, Datadog.
DevSecOps	Trivy, SonarQube, Snyk, OWASP Dependency-Check, Vault, Secrets Manager, Key Vault.

Proyecto típico de DevOps

Código en Git → Pruebas automáticas → Imagen Docker → Registro → Terraform → Kubernetes → Prometheus/Grafana

Qué conviene demostrar en un portafolio

- Pipeline reproducible y documentado.
- Despliegue con rollback.
- Gestión segura de secretos.
- Infraestructura creada desde código.
- Dashboards y alertas para la aplicación.

2. Cloud Engineer

Diseña, implementa y administra infraestructura en AWS, Microsoft Azure o Google Cloud. Su prioridad es construir servicios seguros, escalables, disponibles y eficientes en costos.

Servicios equivalentes más frecuentes

Categoría	AWS	Azure	Google Cloud
Máquinas virtuales	EC2	Azure Virtual Machines	Compute Engine
Almacenamiento de objetos	S3	Blob Storage	Cloud Storage
Red privada	VPC	Virtual Network	VPC
Kubernetes administrado	EKS	AKS	GKE
Base de datos SQL	RDS	Azure SQL	Cloud SQL
Serverless	Lambda	Azure Functions	Cloud Functions
Monitoreo	CloudWatch	Azure Monitor	Cloud Monitoring
Identidades	IAM	Entra ID / RBAC	Cloud IAM

Conocimientos y herramientas

Área	Tecnologías y herramientas
Redes cloud	VPC/VNet, subredes, rutas, NAT, gateways, peering, DNS, CDN y balanceadores.
Seguridad	IAM, roles, políticas, KMS, secretos, cifrado, auditoría y mínimo privilegio.
Automatización	Terraform, CloudFormation, Bicep, Pulumi, Packer y Ansible.
CLI y SDK	AWS CLI, Azure CLI, gcloud y SDK para Python/Go/JavaScript.
Operación	Alta disponibilidad, autoescalado, backups, recuperación ante desastres y FinOps.

Proyecto típico de Cloud

Internet → Balanceador → Subred privada → Aplicación → Base administrada → Backups → Monitoreo

3. Platform Engineer

Construye una plataforma interna de autoservicio para que los equipos de desarrollo creen y desplieguen servicios sin dominar todos los detalles de infraestructura.

Conceptos centrales

- **Internal Developer Platform:** entorno integrado con plantillas, documentación, catálogo de servicios y automatización.
- **Golden paths:** caminos recomendados y seguros para crear aplicaciones de forma consistente.
- **Autoservicio:** los desarrolladores solicitan recursos mediante portales o APIs sin abrir tickets manuales.
- **GitOps:** Git conserva el estado deseado y una herramienta sincroniza ese estado con Kubernetes.
- **Política como código:** reglas automáticas de seguridad, cumplimiento y configuración.

Área	Tecnologías y herramientas
Plataforma	Kubernetes, OpenShift, Rancher, Docker, Helm y Kustomize.
Portales internos	Backstage, Port, Humanitec, Cortex y OpsLevel.
GitOps	Argo CD, Flux CD, GitHub y GitLab.
Aprovisionamiento	Terraform, OpenTofu, Pulumi, Crossplane, Ansible y Packer.
Políticas y seguridad	OPA, Gatekeeper, Kyverno, Vault, External Secrets Operator, cert-manager y Keycloak.
Observabilidad	Prometheus, Grafana, Loki, Tempo, OpenTelemetry y Jaeger.
Desarrollo interno	Go, Python, TypeScript, APIs REST/GraphQL y CLI propias.

Proyecto típico de plataforma

Plantilla de microservicio → Repositorio → Pipeline → Infraestructura → Despliegue → Monitoreo → Documentación

Diferencia con DevOps

DevOps se enfoca en automatizar la entrega y operación de aplicaciones. Platform Engineering convierte esas capacidades en un producto interno reutilizable para muchos equipos.

4. Site Reliability Engineer (SRE)

Aplica prácticas de ingeniería de software a la operación. Busca que los sistemas sean medibles, confiables, resilientes y capaces de recuperarse rápidamente.

Conceptos principales

Área	Tecnologías y herramientas
SLI	Indicador medido, por ejemplo disponibilidad, latencia o tasa de errores.
SLO	Objetivo interno de confiabilidad, por ejemplo 99,9 % de disponibilidad mensual.
SLA	Compromiso contractual o formal con usuarios o clientes.
Error budget	Cantidad de fallas permitida antes de frenar cambios y priorizar confiabilidad.
MTTR	Tiempo medio necesario para recuperar el servicio.
Runbook	Guía operativa paso a paso para responder a incidentes conocidos.
Postmortem	Análisis sin culpables sobre qué ocurrió, por qué y cómo evitar que se repita.

Herramientas frecuentes

Área	Tecnologías y herramientas
Métricas y alertas	Prometheus, Alertmanager, Grafana, Datadog, New Relic y Zabbix.
Logs y trazas	Loki, Elasticsearch/OpenSearch, Fluent Bit, OpenTelemetry, Jaeger, Tempo y Zipkin.
Incidentes	PagerDuty, Opsgenie, incident.io, Jira, Slack y Microsoft Teams.
Carga y rendimiento	k6, Apache JMeter, Locust y Gatling.
Ingeniería del caos	Chaos Mesh, LitmusChaos, Gremlin y AWS Fault Injection Service.
Diagnóstico	kubectl, curl, dig, traceroute, tcpdump, Wireshark, top, htop e iostat.

Ejemplo de SLO

99,9 % disponibilidad → 95 % bajo 300 ms → menos de 1 % de errores 5xx → alertas → runbook

Ciclo de respuesta a incidentes

1	Detectar Una alerta o un reporte indica que el servicio no cumple el comportamiento esperado.
2	Evaluar impacto Determinar usuarios afectados, alcance, severidad y servicios dependientes.
3	Mitigar Restaurar el servicio con rollback, failover, escalado o desactivación temporal de una función.
4	Resolver Identificar y corregir la causa técnica con evidencia de logs, métricas y trazas.
5	Aprender Documentar el incidente, mejorar alertas, automatizar respuestas y actualizar runbooks.

Cuatro señales de oro

- Latencia: tiempo de respuesta.
- Tráfico: volumen de solicitudes o trabajo.
- Errores: solicitudes fallidas o resultados incorrectos.
- Saturación: proximidad al límite de CPU, memoria, disco, conexiones o colas.

5. System Administrator

Administra sistemas operativos, servidores, identidades, servicios, redes, almacenamiento y copias de seguridad. Es una base excelente para evolucionar hacia DevOps, Cloud o SRE.

Áreas y herramientas

Área	Tecnologías y herramientas
Linux	Ubuntu Server, Debian, RHEL, Rocky Linux, AlmaLinux, SUSE; systemd, journalctl, cron, permisos y paquetes.
Windows	Windows Server, PowerShell, Active Directory, Group Policy, DNS, DHCP, IIS, Hyper-V y Windows Admin Center.
Redes	TCP/IP, DNS, DHCP, Nginx, Apache, firewalls, VPN, proxy, balanceo y herramientas de diagnóstico.
Almacenamiento	Particiones, LVM, RAID, NFS, Samba, iSCSI, SAN, NAS, cuotas y sistemas de archivos.
Virtualización	VMware, Hyper-V, KVM, Proxmox y VirtualBox.
Automatización	Ansible, Bash, PowerShell y Python.
Monitoreo	Zabbix, Nagios, Prometheus y Grafana.
Backups	Veeam, BorgBackup, Restic, rsync y políticas de retención.

Proyecto típico de administración

Servidor Linux → Usuarios y SSH → Nginx → Firewall → TLS → Backups → Monitoreo

Competencias operativas

- Instalación y endurecimiento de servidores.
- Gestión de usuarios, permisos y directorios.
- Diagnóstico de servicios, puertos, memoria, CPU y disco.
- Aplicación de parches y mantenimiento preventivo.
- Recuperación a partir de backups probados.

6. Database Administrator (DBA)

Protege la información, mantiene los motores de base de datos, optimiza consultas y garantiza backup, recuperación, disponibilidad y control de acceso.

Motores y lenguajes

Área	Tecnologías y herramientas
Relacionales	PostgreSQL, MySQL, MariaDB, Microsoft SQL Server y Oracle Database.
NoSQL y caché	MongoDB, Redis, Cassandra, DynamoDB y Elasticsearch/OpenSearch.
Lenguajes	SQL, PL/pgSQL, T-SQL y PL/SQL.

Administración y rendimiento

Área	Tecnologías y herramientas
Administración	Instalación, configuración, usuarios, roles, privilegios, esquemas, tablas, vistas y auditoría.
Rendimiento	EXPLAIN/ANALYZE, índices, particionamiento, estadísticas, consultas lentas, bloqueos y connection pooling.
PostgreSQL	pg_stat_activity, pg_stat_statements, VACUUM, ANALYZE, pg_dump, pg_restore y pgBackRest.
Alta disponibilidad	Replicación, failover, read replicas, Patroni, PgBouncer, HAProxy y clusters administrados.
Herramientas gráficas	DBeaver, pgAdmin, MySQL Workbench, SSMS, Oracle SQL Developer y DataGrip.

Proyecto típico de DBA

Aplicación → Pool de conexiones → PostgreSQL principal → Réplica → Backup/PITR → Monitoreo

Relación con DevOps

El DBA moderno colabora con pipelines de migraciones, bases administradas en cloud, observabilidad, automatización y prácticas Database as Code, aunque sigue siendo responsable del rendimiento y la integridad de los datos.

Comparación entre los seis roles

Cada perfil resuelve una necesidad diferente. En organizaciones pequeñas una persona puede asumir varias funciones; en equipos grandes suelen existir especialistas.

Rol	Pregunta principal	Resultado esperado	Herramientas distintivas
DevOps Engineer	¿Cómo automatizamos la entrega?	Pipelines rápidos y reproducibles	GitHub Actions, Jenkins, Docker, Terraform
Cloud Engineer	¿Cómo construimos infraestructura cloud?	Arquitecturas seguras y escalables	AWS/Azure/GCP, VPC, IAM, Terraform
Platform Engineer	¿Cómo facilitamos el trabajo de desarrollo?	Plataforma interna de autoservicio	Kubernetes, Backstage, Argo CD, Crossplane
SRE	¿Cómo mantenemos confiabilidad?	SLO cumplidos y recuperación rápida	Prometheus, Alertmanager, OpenTelemetry, PagerDuty
System Administrator	¿Cómo operamos servidores y servicios?	Sistemas estables, seguros y respaldados	Linux, Windows Server, Ansible, VMware
DBA	¿Cómo protegemos y optimizamos los datos?	Datos disponibles, íntegros y recuperables	PostgreSQL, SQL Server, Oracle, backup/replicación

Zonas de colaboración

- **DevOps + Cloud:** automatización del aprovisionamiento y despliegue en proveedores cloud.
- **DevOps + Platform:** construcción de pipelines y plantillas reutilizables.
- **Platform + SRE:** observabilidad y confiabilidad incorporadas por defecto en la plataforma.
- **SysAdmin + Cloud:** migración de servidores tradicionales a servicios virtuales o administrados.
- **DBA + SRE:** objetivos de latencia, disponibilidad, capacidad y recuperación de datos.

Terraform dentro de Infraestructura y Operaciones

Terraform describe infraestructura mediante código declarativo. Permite revisar cambios, reproducir entornos y mantener una relación controlada entre la configuración y los recursos reales.

Área	Tecnologías y herramientas
DevOps Engineer	Integra terraform plan y apply en pipelines; crea entornos de desarrollo, staging y producción.
Cloud Engineer	Aprovisiona VPC, subredes, máquinas, balanceadores, bases de datos, IAM y almacenamiento.
Platform Engineer	Crea módulos y servicios de autoservicio para equipos de desarrollo.
SRE	Estandariza observabilidad, alertas, entornos de recuperación y recursos resilientes.
System Administrator	Automatiza máquinas virtuales, redes y recursos de infraestructura, especialmente en entornos híbridos.
DBA	Aprovisiona bases administradas, parámetros, réplicas y políticas de backup; no reemplaza la administración SQL.

Flujo básico de trabajo

1	Escribir configuración Definir proveedores, recursos, variables, módulos y salidas en archivos .tf.
2	Inicializar Ejecutar terraform init para descargar proveedores y preparar el proyecto.
3	Validar y planificar Usar terraform fmt, validate y plan para revisar errores y cambios propuestos.
4	Aplicar Ejecutar terraform apply después de una revisión o aprobación automática.

5

Administrar el estado

Guardar el state de forma remota, segura y con bloqueo para trabajo en equipo.

6

Mantener

Revisar drift, actualizar módulos, probar cambios y documentar decisiones.

Terraform no trabaja solo

Normalmente se combina con Git para versionado, CI/CD para automatización, Ansible para configuración, Vault para secretos, Kubernetes para orquestación y herramientas cloud para observabilidad.

Buenas prácticas esenciales

Área	Tecnologías y herramientas
Estado remoto	Usar un backend compartido con cifrado, control de acceso, versionado y bloqueo.
Módulos	Agrupar recursos relacionados y crear interfaces claras mediante variables y outputs.
Revisión	Ejecutar fmt, validate, plan y análisis de seguridad antes de aplicar.
Separación de entornos	Evitar mezclar desarrollo y producción; definir límites y permisos independientes.
Secretos	No guardar credenciales en archivos .tf, variables públicas ni repositorios.
Cambios pequeños	Aplicar modificaciones acotadas, revisables y con estrategia de recuperación.

Ruta de aprendizaje recomendada

La secuencia siguiente evita saltar directamente a Kubernetes o Terraform sin comprender los fundamentos que explican su funcionamiento.

1	Linux y sistemas Terminal, archivos, permisos, procesos, servicios, paquetes, logs y SSH.
2	Redes TCP/IP, subredes, DNS, HTTP/HTTPS, routing, NAT, firewalls y diagnóstico.
3	Git y automatización Repositorios, ramas, pull requests, Bash, PowerShell y Python.
4	Contenedores Docker, imágenes, volúmenes, redes, Compose y registros.
5	CI/CD Pipelines de build, test, análisis, publicación y despliegue.
6	IaC y configuración Terraform/OpenTofu, módulos, state, Ansible y gestión de secretos.
7	Cloud Elegir AWS, Azure o GCP; cómputo, redes, almacenamiento, IAM y bases administradas.
8	Kubernetes Pods, deployments, services, ingress, ConfigMaps, Secrets, Helm y almacenamiento.
9	Observabilidad y SRE Prometheus, Grafana, logs, trazas, alertas, SLO, incidentes y postmortems.
10	Especialización Profundizar en DevOps, Cloud, Platform, SRE, SysAdmin o DBA.

Combinación esencial para una orientación DevOps

Secuencia recomendada

Linux + Redes + Git + Python → Docker + CI/CD → Terraform + Ansible → Cloud + Kubernetes → Prometheus + Grafana + Seguridad

Consejo de estudio

Cada etapa debe cerrar con un proyecto funcional y documentación en Git. La práctica integrada vale más que aprender comandos aislados de memoria.

Evidencias de aprendizaje por etapa

Área	Tecnologías y herramientas
Fundamentos	Administrar un servidor, diagnosticar red y automatizar una tarea con script.
Contenedores y CI/CD	Construir una imagen y desplegarla mediante un pipeline.
IaC y cloud	Crear un entorno reproducible con Terraform y destruirlo de forma controlada.
Kubernetes	Desplegar, actualizar, escalar y diagnosticar una aplicación.
Observabilidad	Crear dashboards, alertas y un procedimiento de respuesta.
Especialización	Resolver un proyecto integral alineado con el rol elegido.

Proyectos prácticos para el portafolio

Los siguientes ejercicios integran varias tecnologías y permiten demostrar capacidades reales.

1	1. Servidor web seguro Crear una VM Linux, configurar usuarios y SSH, instalar Nginx, habilitar HTTPS, firewall, monitoreo y backup.
2	2. Pipeline de aplicación Aplicación simple en GitHub, pruebas, imagen Docker, publicación en registro y despliegue automático.
3	3. Infraestructura con Terraform VPC/VNet, subredes, máquina virtual, balanceador, base administrada y outputs documentados.
4	4. Kubernetes básico Desplegar una aplicación con Deployment, Service, Ingress, ConfigMap, Secret, volúmenes y Helm.
5	5. Observabilidad completa Métricas con Prometheus, dashboards en Grafana, logs con Loki y alertas básicas.
6	6. Plataforma interna mínima Plantilla de repositorio, pipeline reutilizable y despliegue GitOps con Argo CD.
7	7. Base de datos resiliente PostgreSQL con roles, backup, restore probado, monitoreo de consultas y réplica o servicio administrado.
8	8. Simulación de incidente Provocar una falla controlada, detectar con alertas, seguir un runbook y elaborar un postmortem.

Qué debe contener cada proyecto

- README con arquitectura y objetivo.
- Diagrama del flujo.
- Código versionado y estructura clara.
- Pasos de instalación reproducibles.
- Pruebas o validaciones.
- Capturas de dashboards o resultados.
- Sección de problemas encontrados y solución.

Caminos de especialización

No existe una única ruta profesional. Los fundamentos permiten moverse entre perfiles y combinar responsabilidades según la organización.

Área	Tecnologías y herramientas
SysAdmin → DevOps	Agregar Git, scripting, CI/CD, Docker, Terraform y cloud.
SysAdmin → Cloud	Profundizar redes, IAM, automatización, arquitectura y servicios administrados.
DevOps → Platform	Diseñar productos internos, autoservicio, APIs, Kubernetes avanzado y GitOps.
DevOps → SRE	Profundizar observabilidad, SLO, capacidad, performance y respuesta a incidentes.
DBA → Cloud DBA	Bases administradas, automatización, replicación multizona, seguridad y costos.
Cloud → SRE	Arquitectura resiliente, métricas, incidentes, pruebas de carga y caos.

Cómo elegir un rol inicial

- **Te gusta automatizar entregas:** DevOps Engineer.
- **Te interesan redes, arquitectura y proveedores cloud:** Cloud Engineer.
- **Te gusta crear herramientas para otros desarrolladores:** Platform Engineer.
- **Te interesan métricas, incidentes y confiabilidad:** SRE.
- **Te atraen sistemas operativos, servidores y soporte técnico profundo:** System Administrator.
- **Te interesan SQL, rendimiento, integridad y recuperación de datos:** DBA.

Conclusión

Infraestructura y Operaciones es un ecosistema. El profesional más sólido comprende la base común, domina una especialidad y sabe colaborar con las demás.

Glosario esencial

Términos que aparecen con frecuencia en entornos de Infraestructura, Operaciones, DevOps y SRE.

Área	Tecnologías y herramientas
CI/CD	Integración continua y entrega o despliegue continuo.
IaC	Infraestructura como código: recursos definidos en archivos versionables.
HCL	Lenguaje de configuración utilizado principalmente por Terraform.
Container	Paquete aislado que incluye aplicación y dependencias.
Orquestador	Sistema que administra contenedores; Kubernetes es el ejemplo principal.
GitOps	Uso de Git como fuente del estado deseado de aplicaciones o infraestructura.
Observabilidad	Capacidad de comprender el estado interno mediante logs, métricas y trazas.
Drift	Diferencia entre la configuración declarada y el estado real.
Rollback	Regreso a una versión anterior después de un despliegue problemático.
SLO	Objetivo interno y medible de nivel de servicio.
Runbook	Procedimiento documentado para realizar una tarea o responder a un incidente.
MTTR	Tiempo medio de recuperación de un servicio.
HA	Alta disponibilidad mediante redundancia y tolerancia a fallas.
PITR	Recuperación de una base de datos a un punto específico en el tiempo.
FinOps	Prácticas para gestionar y optimizar costos cloud.

Método + automatización + observabilidad + seguridad = operaciones confiables

Fin de la guía

Resumen final

Infraestructura y Operaciones sostiene todo el ciclo de vida del software. Linux y redes explican el entorno; Git y automatización controlan los cambios; Terraform y cloud crean recursos; Kubernetes ejecuta aplicaciones; observabilidad y SRE mantienen la confiabilidad; seguridad y bases de datos protegen el servicio y la información.

DEVOPS • CLOUD • PLATFORM • SRE • SYSTEM ADMINISTRATION • DBA